DATA SCIENCE

PRT ADVANCED STUDY GUIDE

Python · SQL

■ Python (Primary)

■■ SQL (Supporting)

■ SECTION 1 — PYTHON CORE

1.1 Comprehensions & Functional Tools

These are tested frequently at advanced level. Know when to use each form.

- List / Dict / Set comprehensions with conditions
- **lambda** — anonymous one-liner functions
- **map(func, iterable)** — apply function element-wise
- **filter(func, iterable)** — keep elements where func returns True
- **reduce(func, iterable)** — from functools; fold to single value

```
squares = [x**2 for x in range(10) if x % 2 == 0]
```

■ *TIP: Comprehensions are faster than for-loops in most cases.*

1.2 Decorators & Generators

- Decorator = function that wraps another function → @decorator syntax
- Use cases: logging, timing, authentication
- Generator = yields one value at a time → memory efficient
- **yield** vs **return** — generator pauses, return exits

```
def timer(func): def wrapper(*args): import time; t=time.time() result=func(*args)
print(time.time()-t); return result return wrapper
```

1.3 Object-Oriented Programming (OOP)

- **Class / __init__ / self** — blueprint & constructor
- Inheritance — child class inherits parent methods
- Polymorphism — same method name, different behaviour
- Encapsulation — private attrs with _ or __
- Special methods: __str__, __len__, __repr__

■ *Common mistake: forgetting **self** as first param in methods.*

1.4 Exception Handling

- try / except / else / finally block structure
- Raise custom exceptions: class MyError(Exception): pass
- Multiple except clauses for different error types
- Use finally for cleanup (closing files, DB connections)

■ SECTION 2 — NUMPY

2.1 Array Fundamentals

- np.array(), np.zeros(), np.ones(), np.arange(), np.linspace()
- ndarray.shape, .dtype, .ndim, .size
- Indexing: arr[0], arr[1:4], arr[::2]
- Boolean indexing: arr[arr > 5]
- Fancy indexing: arr[[0, 2, 4]]

2.2 Broadcasting & Vectorisation

- Operations on arrays of compatible shapes without loops
- Shape rules: dimensions align from the right; size 1 is broadcast
- Vectorised ops are 10-100x faster than Python loops

■ *TIP: Always prefer np operations over Python for-loops on arrays.*

2.3 Reshaping & Stacking

- reshape(rows, cols) — change shape; -1 infers dimension
- np.stack(), np.vstack(), np.hstack(), np.concatenate()
- np.flatten() vs np.ravel() — copy vs view

2.4 Statistical Functions

Function	Description
np.mean(a, axis=0)	Mean along axis
np.std(a)	Standard deviation
np.percentile(a,75)	75th percentile
np.median(a)	Median value
np.corrcoef(x,y)	Correlation matrix

■ SECTION 3 — PANDAS ■ Most Important

3.1 Core Structures

- Series = 1-D labelled array
- DataFrame = 2-D table (rows × columns)
- df.head(), df.tail(), df.info(), df.describe()
- df.shape, df.columns, df.dtypes, df.index

3.2 Selection & Filtering

- df['col'] or df[['col1','col2']] — column select
- df.loc[row_label, col_label] — label-based
- df.iloc[row_idx, col_idx] — integer-based
- Boolean filter: df[df['age'] > 25]
- Multiple conditions: df[(df.a > 1) & (df.b < 10)]

3.3 groupby & Aggregation

```
df.groupby('dept')['salary'].agg(['mean','max','count'])
```

- Multiple group keys: groupby(['dept','year'])
- transform() — returns same-shape result (good for filling)
- Named aggregations: .agg(avg_sal=('salary','mean'))

■ *HIGH PRIORITY — groupby appears in almost every advanced test.*

3.4 merge & join

Type	How
inner	Only matching rows in both
left	All left rows + matched right
right	All right rows + matched left
outer	All rows from both

```
pd.merge(df1, df2, on='id', how='left')
```

3.5 apply, map, applymap

- df['col'].apply(func) — row-wise on a column
- df.apply(func, axis=1) — across each row
- df['col'].map(dict_or_func) — element-wise map

3.6 Handling Missing Values

- df.isnull().sum() — count NaNs per column
- df.dropna(subset=['col']) — drop rows with NaN
- df.fillna(value) or df.fillna(method='ffill')
- df.interpolate() — fill with interpolated values

3.7 pivot_table

```
pd.pivot_table(df, values='sales', index='region', columns='quarter', aggfunc='sum',
fill_value=0)
```

■ SECTION 4 — STATISTICS & MACHINE LEARNING

4.1 Descriptive Statistics

Term	Formula / Note
Mean	Sum / n
Median	Middle value when sorted
Variance	Avg squared deviation from mean
Std Dev	√Variance
Skewness	> 0 right-skewed, < 0 left-skewed
Kurtosis	Tail heaviness; normal = 3

4.2 Scikit-learn Pipeline

- from sklearn.model_selection import train_test_split
- X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
- Scaling: StandardScaler (z-score) | MinMaxScaler (0-1)
- Cross-validation: cross_val_score(model, X, y, cv=5)

4.3 Key Algorithms

Algorithm	Type	Key Params
Linear Regression	Regression	fit_intercept
Logistic Regression	Classification	C, max_iter
Decision Tree	Both	max_depth, min_samples_split
Random Forest	Both	n_estimators, max_depth
K-Nearest Neighbors	Both	n_neighbors, metric

4.4 Model Evaluation Metrics

Metric	Formula	Good for
Accuracy	(TP+TN)/Total	Balanced classes
Precision	TP/(TP+FP)	Minimise false positives
Recall	TP/(TP+FN)	Minimise false negatives
F1 Score	2·P·R/(P+R)	Imbalanced classes
ROC-AUC	Area under ROC curve	Overall classifier quality
MSE/RMSE	Mean squared error	Regression tasks
R^2 Score	Variance explained ratio	Regression tasks

■ *Know ALL metrics — confusion matrix questions are very common.*

■ SECTION 5 — DATA VISUALISATION

5.1 Matplotlib

- plt.plot(), plt.bar(), plt.hist(), plt.scatter()
- plt.subplots(nrows, ncols) — multi-panel figures
- ax.set_title(), ax.set_xlabel(), ax.legend()
- plt.tight_layout() — prevent overlapping
- plt.savefig('fig.png', dpi=150, bbox_inches='tight')

5.2 Seaborn

Plot	Function	Use
Heatmap	sns.heatmap(df.corr())	Correlation matrix
Pairplot	sns.pairplot(df)	Feature relationships
Boxplot	sns.boxplot(x,y,data=df)	Outlier detection
Violin	sns.violinplot()	Distribution shape
Count plot	sns.countplot(x, data=df)	Category frequency
Dist plot	sns.histplot()	Distribution

■■ SECTION 6 — SQL (Advanced)

6.1 Core Clauses

```
SELECT col1, col2 FROM table WHERE condition GROUP BY col1 HAVING aggregate_condition
ORDER BY col2 DESC LIMIT 10;
```

6.2 JOINs

JOIN Type	Returns
INNER JOIN	Only matching rows in BOTH tables
LEFT JOIN	All left rows + matched right (NULLs if no match)
RIGHT JOIN	All right rows + matched left
FULL OUTER	All rows from both; NULLs for no match
SELF JOIN	Table joined with itself (e.g. manager-employee)
CROSS JOIN	Cartesian product of both tables

6.3 Window Functions ■■■

■ *HIGHEST PRIORITY — always in advanced SQL tests.*

- ROW_NUMBER() — unique sequential integer per row
- RANK() — same rank for ties, gaps after ties
- DENSE_RANK() — same rank for ties, NO gaps
- LEAD(col, n) — value n rows ahead
- LAG(col, n) — value n rows behind
- SUM/AVG OVER (PARTITION BY ... ORDER BY ...)

```
SELECT name, salary, RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk FROM
employees;
```

6.4 CTEs & Subqueries

```
WITH high_earners AS ( SELECT * FROM emp WHERE salary > 80000 ) SELECT dept, COUNT(*)
FROM high_earners GROUP BY dept;
```

- CTE = Common Table Expression — cleaner than nested subqueries
- Correlated subquery — inner query references outer query column
- EXISTS vs IN — EXISTS is faster on large tables

6.5 CASE WHEN & NULL Handling

```
SELECT name, CASE WHEN score >= 90 THEN 'A' WHEN score >= 75 THEN 'B' ELSE 'C' END AS
grade, COALESCE(phone, 'N/A') AS contact FROM students;
```

6.6 Python + SQL Integration

- import sqlite3; conn = sqlite3.connect('db.sqlite3')
- pd.read_sql('SELECT * FROM table', conn) — into DataFrame
- SQLAlchemy — ORM for production database connections
- df.to_sql('table', conn, if_exists='replace') — write DF to DB

■ SECTION 7 — QUICK REVISION CHEAT SHEET

Priority Matrix

Topic	Section	Priority
Pandas groupby + merge	3.3/3.4	■■■
Window Functions (SQL)	6.3	■■■
ML Evaluation Metrics	4.4	■■■
sklearn Pipeline	4.2	■■■
Feature Engineering & Scaling	4.2	■■■
Decorators & Generators	1.2	■■
OOP in Python	1.3	■■
CTEs & Subqueries	6.4	■■
NumPy Broadcasting	2.2	■■
Seaborn Plots	5.2	■

Last-Minute Must-Know Facts

★ loc uses labels, iloc uses integers
★ groupby().agg() can take a dict of {col: func}
★ HAVING filters aggregated results; WHERE filters rows before aggregation
★ ROC-AUC = 0.5 means random classifier; 1.0 = perfect
★ Precision-Recall tradeoff — improving one usually hurts the other
★ StandardScaler = zero mean, unit variance | MinMaxScaler = [0,1] range
★ Random Forest = ensemble of Decision Trees with bagging
★ Overfitting signs: high train acc, low test acc → reduce complexity
★ pd.concat([df1,df2], axis=0) stacks rows; axis=1 stacks columns
★ NaN arithmetic always returns NaN — handle missing before modelling

---

All the best for your PRT Test! — Prepared for Syam Sundhar